

Operators & Expressions

Arithmetic · Comparison · Logical · Ternary · Nullish

HOW TO USE THIS DOCUMENT

- Part 1** → Keep open during class. Follow the code on instructor's screen.
- In 'think it' rows: predict output BEFORE it runs.
- Part 2** → Work on Hands-On tasks during exercise block (15 min).
- Part 3** → Use Visual Summary after class to review and remember.

TODAY'S TOPICS

- Topic 1** Arithmetic Operators
- Topic 2** Assignment Operators
- Topic 3** Comparison — == vs ===
- Topic 4** Logical — && || !
- Topic 5** Ternary — condition ? a : b
- Topic 6** Optional Chaining ?. & ??

PART 1 — FOLLOW ALONG WITH YOUR INSTRUCTOR

Topic 1

Arithmetic Operators

+ - * / % ** ++ --

VISUAL REFERENCE

+	add	10 + 3 → 13	-	subtract	10 - 3 → 7
*	multiply	10 * 3 → 30	/	divide	10 / 3 → 3.33...
%	modulo (remainder)	10 % 3 → 1	**	exponent	2 ** 8 → 256
++	increment	x++ adds 1 to x	--	decrement	x-- subtracts 1

FOLLOW ALONG — predict each output!

<code>console.log(10 + 3);</code>	→	your answer →
<code>console.log(10 % 3);</code>	→	your answer →
<code>console.log(2 ** 8);</code>	→	your answer →
<code>console.log(42 % 2 === 0);</code>	→	your answer →

Topic 2

Assianment Operators

Shortcuts: += -= *= /=

VISUAL REFERENCE

<code>x += 5</code>	same as	<code>x = x + 5</code>
<code>x -= 3</code>	same as	<code>x = x - 3</code>
<code>x *= 2</code>	same as	<code>x = x * 2</code>
<code>x /= 4</code>	same as	<code>x = x / 4</code>

Topic 3

Comparison — == vs === (CRITICAL)

Always use === to avoid type coercion surprises

VISUAL REFERENCE

==

Loose equality — DOES type coercion

5 == '5' is TRUE ■

===

===

Strict equality — NO coercion, checks value AND type

5 === '5' is FALSE ■ — USE THIS ALWAYS

FOLLOW ALONG — guess before it runs!

console.log(5 == "5");



your answer →

console.log(5 === "5");



your answer →

console.log(0 == false);



your answer →

console.log(0 === false);



your answer →

console.log(null == undefined);



your answer →

Topic 4

Logical Operators — && || !

AND, OR, NOT — combine boolean expressions

VISUAL REFERENCE

&&

AND

BOTH must be true

true && true → true

FOLLOW ALONG

console.log(true && false);



your answer →

console.log(5 > 2 && 10 < 20);



your answer →

console.log(!true);



your answer →

||

OR

EITHER can be true

true || false → true

!

NOT

Flips the boolean

!true → false

Topic 5

Ternary Operator

condition ? valueIfTrue : valueIfFalse

VISUAL REFERENCE

```
age >= 18 ? "adult" : "minor"
```

condition

if TRUE →

if FALSE →

FOLLOW ALONG

```
const status = age >= 18 ? "adult" : "minor";
```



your answer →

```
const result = score >= 50 ? "pass" : "fail";
```



your answer →

Topic 6

Optional Chaining ?. & Nullish Coalescing ??

Modern safety features (2020)

VISUAL REFERENCE

?.

Optional Chaining

Safely access nested properties.

Returns undefined instead of error.

```
user?.address?.city
```

without ?. → TypeError crash!

??

Nullish Coalescing

Default ONLY when null/undefined.

Respects 0, "", false as valid.

```
count ?? 10
```

(unlike ||, which treats 0 as empty)

FOLLOW ALONG

```
console.log(user.address?.city);
```



your answer →

```
let count = 0;
```



your answer →

```
console.log(count ?? 10);
```



your answer →

```
console.log(count || 10);
```



your answer →

PART 2 — HANDS-ON EXERCISES (15 minutes)

Open index.js or JSFiddle. Predict output BEFORE running — that's where learning happens.

Task 1

Arithmetic Detective

- Declare two const variables: `a = 17` and `b = 5`.
- Log each arithmetic result: `a + b`, `a - b`, `a * b`, `a / b`, `a % b`, `a ** b`.
- Before running, write a comment predicting each output.
- Bonus: use `%` to check if the number 42 is even or odd (log `'even'` or `'odd'`).

Task 2

Equality Trap

- Predict the output of each expression BEFORE running — write your guess as a comment:
- `5 == "5"` | `5 === "5"` | `0 == false` | `0 === false` | `null == undefined`
- Run each and compare with your predictions.
- In a comment, explain WHY you should always prefer `===` over `==`.

Task 3

Access Control Logic

- Create these variables: `const age = 19`; `const hasLicense = true`; `const hasCar = false`;
- Using `&&` and `||`, log whether the person:
- (a) can drive — needs `age >= 18` AND `hasLicense`
- (b) can travel — needs `hasLicense` OR `hasCar`
- Try flipping `hasLicense` to `false` and re-run. What changes?

Bonus

Ternary & Nullish

- Write a ternary: `const mood = age >= 18 ? 'adult' : 'minor'`; — then log it.
- Declare: `let username = null`; `const display = username ?? 'Guest'`; — log `display`.
- Now try: `const display2 = username || 'Guest'`; — does it behave the same?
- In a comment, explain what `??` does that `||` doesn't.

PART 3 — VISUAL SUMMARY (review after class)

ARITHMETIC

COMPARISON

+	-	*	/	basic math
%				remainder
**				exponent
++	--			±1 increment
+=	-=	*=	/=	assign shortcuts

==	"5" == 5 → TRUE ■
===	"5" === 5 → FALSE ■
!==	strict not-equal
> < >= <=	basic compare
Rule: ALWAYS use === / !==	

LOGICAL

&&	AND — both must be true	true && false → false
	OR — either can be true	true false → true
!	NOT — flips the boolean	!true → false

TERNARY & NULLISH

condition ? a : b	choose between two values
user?.address?.city	safe nested access
value ?? 'default'	default only if null/undef
0 ?? 10 → 0	respects 0
0 10 → 10	treats 0 as empty